GummyWars — Documentación técnica

Oct 16, 2026 · @Fabrizio Giovanni Pianarosa

Resumen

GummyWars es un shooter 2.5D por equipos para Roblox: USA contra Vietnam, 4 contra 4 (completado con bots), al mejor de 3 rondas, con soldados de gomita sobre un mapa de bloques de caramelo. Cada ronda alterna un interludio de construcción y tienda con una misión elegida al azar entre cinco tipos.

Este documento describe la arquitectura, las responsabilidades de cada módulo y los contratos entre cliente y servidor del place actual (placeId 77780345632578). Esta pestaña no reproduce código fuente; la copia completa y textual del código está en Anexo — Código fuente.

Dato	Valor
Scripts de juego	2 scripts de arranque + 47 ModuleScripts
Tamaño aproximado	~17.000 líneas de Luau
Equipos	USA (base izquierda) · Vietnam (base derecha)
Formato de partida	3 rondas de 5 min, interludio de 60 s antes de cada una
Misiones	Asesinato, Rey de la Colina, Radio, Captura de bandera, Demolición
Clases	Soldado y Pyro
Persistencia	DataStore `GummyWars_Profiles_v1` (solo si el jugador vio la intro)

Convenciones del proyecto

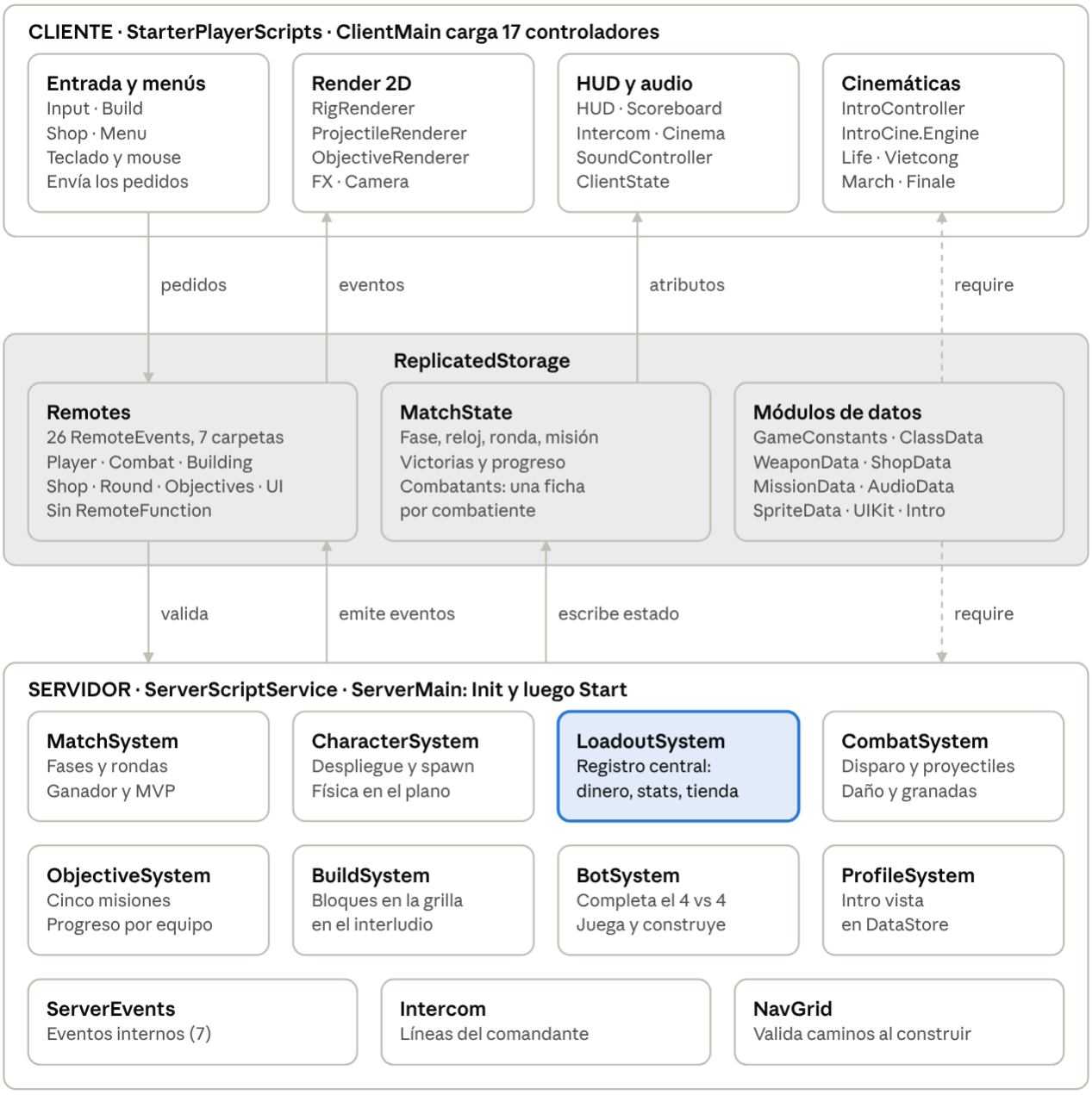
- Patrón de arranque Init/Start: un único script carga todos los módulos de su lado, llama `Init` a cada uno (preparar datos) y luego `Start` en hilos separados (arrancar lógica). Un error en un módulo se registra con `warn` y no detiene al resto.
- El servidor es la autoridad: el cliente pide (disparar, construir, comprar) y el servidor valida y replica.

- El estado compartido viaja por atributos: la partida en `ReplicatedStorage.MatchState`, cada combatiente en una `Configuration` dentro de `MatchState.Combatants`, y el dinero o equipo del jugador en atributos del propio `Player`.
- El mundo es una grilla de bloques de 4 studs sobre el plano Z = 0; columnas y filas se guardan como atributos `Col` y `Row`.
- Comentarios y textos en español.

Arquitectura general

El juego sigue un modelo de servidor autoritativo en tres capas: el cliente captura la entrada y dibuja, `ReplicatedStorage` es el contrato compartido y el servidor decide todo lo que afecta al resultado.

El cliente pide y dibuja; el servidor decide y replica



arquitectura cliente-servidor · 3 capas

Los pedidos del jugador bajan por los remotes y el servidor los valida; los resultados suben como eventos (lo instantáneo) o atributos de `MatchState` (lo continuo), y ambos lados leen los mismos módulos de datos. Dentro del servidor, LoadoutSystem es el registro del que dependen todos los demás, y los sistemas se avisan por ServerEvents en lugar de llamarse en círculo.

Estructura del árbol del juego

El código vive en tres lugares: `ServerScriptService` (lógica autoritativa), `StarterPlayerScripts` (presentación y entrada) y `ReplicatedStorage` (datos y utilidades que ambos lados leen). El resto del árbol son contenedores de mundo, plantillas y audio.

Ubicación	Contenido	Rol
`ServerScriptService.ServerMain.server`	Script de arranque	Carga e inicia todos los módulos de `Systems`
`ServerScriptService.Systems`	8 módulos: Match, Character, Loadout, Combat, Objective, Build, Bot, Profile	Lógica de juego del servidor
`ServerScriptService.Services`	ServerEvents, Intercom, NavGrid	Servicios internos que usan los sistemas
`StarterPlayerScripts.ClientMain`	LocalScript de arranque	Carga 17 controladores en orden fijo
`StarterPlayerScripts.Controllers`	17 controladores + carpeta `IntroCine` (Engine, Life, Vietcong, March)	Render, UI, cámara, entrada, audio, cinemáticas
`ReplicatedStorage.Constants`	GameConstants	Constantes globales de la partida
`ReplicatedStorage.Modules`	11 módulos de datos y utilidades	Datos de armas, clases, tienda, misiones, audio, sprites, UI
`ReplicatedStorage.Remotes`	26 eventos en 7 carpetas	Comunicación cliente-servidor

Ubicación	Contenido	Rol
`ReplicatedStorage.MatchState`	Atributos de la partida + carpeta `Combatants`	Estado replicado
`StarterPlayer.StarterCharacter`	Humanoid + HumanoidRootPart	Personaje mínimo: solo hitbox
`StarterCharacterScripts`	Animate y Health vacíos	Anulan la animación 3D y la regeneración por defecto
`ServerStorage.CharacterTemplates.GummyRig`	Humanoid + HumanoidRootPart	Plantilla que clonan los bots
`ServerStorage.Debug`	MapBuilder, TestSystem, prototipos viejos	Herramientas de desarrollo, no se ejecutan en juego
`ServerStorage.Backups`	Versiones anteriores de la intro	Respaldo, no se ejecutan
`Workspace.World.ActiveMap.WarGummyMap01`	Mapa activo: 411 bloques estáticos, spawns, zonas, decorado	Escenario de juego
`Workspace.World.Runtime`	PlayerBuiltBlocks, Objectives, Projectiles, PlaneAnchor	Objetos creados durante la partida
`Workspace.World.Characters`	Modelos de bots	Personajes no jugadores
`Teams`	USA, Vietnam	Equipos de Roblox
`SoundService`	Music, SFX y grupos de la intro	Audio

El mapa "Frente del Caramelo"

- Simétrico respecto de la columna 0. Cada columna mide 4 studs; la fila 0 es la superficie del suelo.

- USA aparece a la izquierda (lado −1) y Vietnam a la derecha (lado +1), con 3 `SpawnLocation` por bando.
- Zonas de construcción: columnas −50 a −30 para USA y 30 a 50 para Vietnam, filas 1 a 7.
- `Zones.RestrictedZones` contiene las barreras que separan a los equipos durante el interludio.
- `MapBuilder` regenera el mapa desde la barra de comandos de Studio.

Grupos de colisión

CharacterSystem registra `Barrier` y, por equipo, `Char<Equipo>`, `Built<Equipo>` y `Proj<Equipo>`. Los personajes nunca chocan entre sí. Las barreras solo frenan personajes. Los proyectiles de un equipo atraviesan los bloques que construyó ese equipo, que funcionan como aspilleras.

Sistemas del servidor

Ocho sistemas autoritativos comparten un registro central de combatientes (LoadoutSystem) y se avisan entre sí por eventos internos, sin depender unos de otros en círculo. `ServerMain` los carga sin orden fijo; por eso ningún `Init` depende de otro sistema.

Sistema	Responsabilidad	Entradas	Salidas	Depende de
MatchSystem	Máquina de fases de la partida, rondas, ganador y MVP	Evento interno `Deployed`; atributos de prueba `SkipPhase` y `ForceMissions`	Atributos de `MatchState`; remote `Round.Event`; evento interno `PhaseChanged`; líneas de intercom	Character, Loadout, Objective, Build, Intercom
CharacterSystem	Despliegue desde el menú, aparición y reaparición, física 2D del personaje, barreras del interludio	Remotes `Deploy`, `ReturnToMenu`, `SelectClass`, `SetReady`	Atributos del modelo; eventos internos `CharacterReady` y `Deployed`	Loadout, ClassData
LoadoutSystem	Registro de combatientes (humanos y bots), dinero, estadísticas, tienda y equipo	Remotes `Shop.Buy`, `Shop.Equip`	`Configuration` por combatiente en `MatchState.Combatants`; atributos del `Player`; remote `Shop.Notice`	ClassData, ShopData, WeaponData
CombatSystem	Disparo, munición, recarga, proyectiles, headshots, quemaduras, granadas, daño a estructuras y bajas	Remotes `Fire`, `Aim`, `Reload`, `Grenade`	Remotes `Projectile`, `Hit`, `KillFeed`, `GrenadeThrown`, `Explosion`, `Knockback`, `Structure`; eventos internos `Damaged`, `Killed`, `StructureDamaged`, `StructureDestroyed`	Loadout, WeaponData, CombatUtil
ObjectiveSystem	Las cinco misiones: marcadores, progreso de 0 a 1 por equipo y ganador	Eventos internos de combate; posición de personajes	Marcadores en `World.Runtime.Objectives`; atributos `Progress<Equipo>`; remote `Objectives.Event`	Loadout, Intercom

Sistema	Responsabilidad	Entradas	Salidas	Depende de
BuildSystem	Bloques de caramelo en la grilla durante el interludio	Remotes `Building.Place`, `Building.RemoveBlock`	Partes en `World.Runtime.PlayerBuiltBlocks`	Loadout, NavGrid, SpriteData
BotSystem	Completa cada equipo hasta 4 con bots que juegan el objetivo, compran y construyen	Evento interno `PhaseChanged`	Modelos en `World.Characters`; usa la misma API de combate que los jugadores	Character, Combat, Loadout, Build, Objective
ProfileSystem	Perfil persistente: recuerda si el jugador ya vio la intro	Remote `IntroSeen`	Atributos `SeenIntro` y `ProfileLoaded` del `Player`; DataStore	—

Servicios internos

- **ServerEvents**: siete `BindableEvent` compartidos (`PhaseChanged`, `Deployed`, `CharacterReady`, `Damaged`, `Killed`, `StructureDamaged`, `StructureDestroyed`). Desacoplan, por ejemplo, el combate de las misiones.
- **Intercom**: envía claves de diálogo del comandante a un equipo, a ambos o a un jugador, con un tiempo mínimo entre repeticiones. El texto lo elige el cliente.
- **NavGrid**: búsqueda en anchura sobre la grilla de bloques con las mismas reglas de salto que los bots. Valida que un bloque nuevo no corte el camino entre bases y la colina central.

Detalles de cada sistema

- **Combate.** Los proyectiles son simulados en el servidor con spherecasts sobre el plano 2D en cada `Heartbeat`. El cliente solo envía los ángulos ya dispersados; el servidor controla cadencia (con 15 % de tolerancia), munición, recarga y cantidad de perdigones. Un headshot mata en el acto. El daño solo se aplica en la fase `Round`; en `Prep` y `Waiting` se puede disparar sin daño. La granada ("Dona Explosiva") rebota, explota a los 1,7 s o al tocar un enemigo, daña bloques y empuja al lanzador sin herirlo.
- **Personajes.** El personaje es solo `HumanoidRootPart` + `Humanoid`; todo lo visual lo dibuja cada cliente. Un `PlaneConstraint` y un `AlignOrientation` lo fijan al plano Z = 0, y un control en `Heartbeat` lo devuelve si se sale. Al aparecer se escriben en el modelo los atributos `Key`, `Team`, `Class`, `Weapon`, `Ammo`, `MaxAmmo`, `Grenades`, `Aim`, `Reloading`, `Burning`, entre otros.
- **Economía.** 500 de dinero inicial; 100 por baja (+50 por headshot), 150 por objetivo, 600 por ronda ganada y 300 por ronda perdida. Todo dinero positivo suma al puntaje.
- **Construcción.** Un bloque se acepta si está en la zona del equipo, la celda está libre, tiene apoyo, deja dos celdas libres arriba, no forma un túnel de una celda y no corta caminos (NavGrid). Cada bloque tiene 160 de vida y se oscurece al dañarse. Se recargan 12 bloques por interludio.

- **Bots.** Piensan cada 0,12 s y reaccionan a los 0,3 s de ver al enemigo. Tienen rol de ataque o defensa, error de puntería variable, adelantan el tiro según la velocidad del blanco y navegan con búsqueda en anchura sobre la grilla. Rompen bloques enemigos que les cierran el paso y tiran granadas entre 12 y 40 studs.

Sistemas del cliente

El cliente no decide nada del resultado: dibuja el juego en 2D a partir de atributos y remotes, captura la entrada y la envía al servidor. `ClientMain` carga los 17 controladores en este orden fijo, porque varios dependen de los anteriores: ClientState, SoundController, FXController, RigRenderer, CinemaController, ProjectileRenderer, ObjectiveRenderer, CameraController, InputController, BuildController, IntercomController, ShopController, FinaleController, HUDController, ScoreboardController, IntroController, MenuController.

Controlador	Responsabilidad	Escucha	Envía
ClientState	Estado compartido del cliente: ángulo de apuntado, mouse en el plano, sacudida, menú o UI abierta, modo construcción, cinemática en curso, ajustes de música, efectos y sacudida	—	—
SoundController	Sonido 2D con paneo y atenuación según la cámara; música con fundidos	—	—
FXController	Partículas 2D sobre el plano: goma, chispas, humo, fuego, escombros, casquillos, manchas, destellos, textos; sacudida de pantalla	—	—

Controlador	Responsabilidad	Escucha	Envía
RigRenderer	Dibuja cada personaje como un rig 2D en un `BillboardGui` (torso, cabeza, casco, manos, arma) animado cada frame: apuntado, caminata, squash & stretch, recarga, retroceso, muerte	Atributos del modelo	—
CinemaController	Look de película: corrección de color, grano, viñeta, cenizas, artillería lejana, barras de cine, plano de presentación de ronda, cámara de muerte	`Round.Event`, `Combat.Explosion`	—
ProjectileRenderer	Visual de balas, perdigones, trazadoras, llamas, granadas, explosiones e impactos	`Projectile`, `GrenadeThrown`, `Explosion`, `Structure`	—
ObjectiveRenderer	Dibuja bandera, base de bandera, colina, radio y depósito desde los marcadores del servidor; flechas en el borde de pantalla	Marcadores en `World.Runtime.Objectives`	—
CameraController	Cámara lateral que sigue al jugador y se adelanta hacia el mouse; sacudidas, empujones y paneo del menú	ClientState	—

Controlador	Responsabilidad	Escucha	Envía
InputController	A/D mover, Espacio o W saltar, mouse apuntar, click disparar, R recargar, G granada; en el interludio B construir, T tienda, F listo	`Knockback`	`Fire`, `Aim`, `Reload`, `Grenade`, `SelectClass`
BuildController	Modo construcción: bloque fantasma en la grilla, click izquierdo coloca, click derecho quita	Atributos de fase	`Building.Place`, `Building.RemoveBlock`
IntercomController	Retrato del comandante con 3 frames de boca, texto tipo máquina de escribir, estática y voz de bips	`UI.Intercom`	—
ShopController	Intendencia (tienda), solo durante el interludio	Atributos del jugador	`Shop.Buy`, `Shop.Equip`
FinaleController	Cinemática de fin de partida protagonizada por los 3 mejores del bando ganador	Datos del final que manda `Round.Event`	—
HUDController	Reloj, marcador, progreso del objetivo, vida, arma y munición, panel del interludio, killfeed, anuncios, mira con hitmarker, viñeta de daño, pantalla de muerte, resultados y música de fase	`Round.Event`, `Hit`, `KillFeed`, `Shop.Notice`, `Objectives.Event`	`SetReady`
ScoreboardController	Tabla de puntajes mientras se mantiene TAB	`MatchState.Combatants`	—

Controlador	Responsabilidad	Escucha	Envía
IntroController	Director de las cinemáticas de intro: elige una al azar sin repetir la anterior; se salta manteniendo Espacio o B del mando	Atributos de perfil	`IntroSeen`
MenuController	Menú principal, elección de bando y clase, cómo jugar, ajustes, créditos y menú de pausa (M)	Atributos del jugador	`Deploy`, `ReturnToMenu`

Cinemáticas

Todas se dibujan en pixel art 2D dentro de un `ScreenGui`, sobre un espacio de diseño fijo.

- **IntroCine.Engine** es el motor común: cámara con paneo, zoom, rotación y cámara en mano; capas con parallax; actores gomita animados (caminar, saludar, apuntar, saltar, volar por una explosión); partículas, fuego, humo, explosiones y trazadoras; sol, corrección de color, grano, barras y efecto Super 8; subtítulos, carteles, globos de diálogo; música y efectos con fundidos y cámara lenta.
- **Life — "Una vida dulce"**: la vida de Danny Gummerson antes de la guerra, de la infancia a la carta de reclutamiento y Vietnam.
- **Vietcong — "La otra orilla"**: la guerra desde el lado vietnamita, la familia de Lam, el napalm y los túneles.
- **March — "La marcha"**: la columna marcha al frente mientras el Coronel McGummy presenta la guerra y las misiones.
- **FinaleController** tiene dos finales: si gana USA, "Operación Libertad Dulce", que termina en tapa de diario; si gana Vietnam, "La caída de Gomigón", que termina en afiche de propaganda.
- En Studio, IntroController ofrece un panel de depuración para ver cada intro desde cualquier plano y un modo tráiler que oculta la interfaz.

Módulos compartidos y datos de configuración

Todo el balance del juego está en módulos de datos de `ReplicatedStorage`, así que se ajusta sin tocar la lógica. Servidor y cliente leen los mismos valores.

Módulo	Contenido	Lo usan
GameConstants	Equipos y colores, comandantes, reglas de partida, grilla, zonas de construcción, medidas del personaje, cámara, tags, lista de misiones	Ambos lados
ClassData	Clases Soldado y Pyro: vida, velocidad, salto, arma por defecto, armas permitidas, granadas	Ambos lados
WeaponData	Seis armas y la granada, con daño, cadencia, dispersión, alcance, cargador, recarga, retroceso, empuje y daño a bloques	Ambos lados
ShopData	Nueve artículos en tres categorías: armas, equipo y suministros	Loadout, Bot, Shop
MissionData	Nombre e ícono de cada misión y todas las líneas de intercom por equipo	Cliente (HUD, Intercom, Menú, Cine)
CombatUtil	Geometría del plano 2D (ángulo a dirección, hombro, boca del arma), búsqueda de personajes por tag, compañeros de equipo, nombres de grupos de colisión	Ambos lados
AudioData	IDs, volumen y tono de 25 efectos y 7 pistas de música	SoundController, Engine
SpriteData	Sprites 2D y armado de rigs, en unidades de rig mirando a la derecha con origen en los pies	Render y UI
UIKit	Kit de UI "militar de caramelo": paneles, etiquetas, botones, tweens, franjas de peligro, vista previa de rig	Cliente
PixelFont	Tipografía pixel propia. Convierte cualquier `TextLabel` respetando color, transparencia, alineación, escalado, ajuste de línea y color en RichText	Cliente
IntroData	Modo de reproducción de la intro, peso de cada cinemática, canción de "Una vida dulce", música, efectos y líneas del Coronel	Intro

Módulo	Contenido	Lo usan
IntroSprites	Atlas de sprites de las cinemáticas, generado automáticamente	Engine

Reglas de partida (GameConstants.Match)

Parámetro	Valor
Rondas	3 (gana quien llega a 2)
Duración de ronda	300 s
Interludio	60 s (pasa a 3 s si todos los humanos marcan listo)
Fin de ronda	9 s
Fin de partida	58 s (cinemática + resultados)
Tamaño de equipo	4, completado con bots
Reaparición	4 s (1 s en el interludio)
Protección al aparecer	2 s
Bloques por interludio	12

Armas

Arma	Clase	Daño	Cadencia (s)	Cargador	Alcance (studs)	Particularidad	Precio
Fusil de Cerrojo M1	Soldado	70	1,15	5	240	Cerrojo entre disparos	Inicial
Escopeta Recortada	Soldado, Pyro	15 × 7	0,8	6	55	7 perdigones	450
M60 Ametralladora	Soldado	25	0,075	75	180	Automática, −15 % velocidad	650
Rifle de Francotirador	Soldado	95	1,15	5	420	Sin dispersión	700
Lanzallamas	Pyro	7	0,06	80	20	Quema 7 cada 0,4 s durante 3 s	Inicial
Lanzanapalm	Pyro	8	0,06	90	27	Quema 10 cada 0,4 s durante 4,5 s	600
Dona Explosiva	Granada	120	0,8	—	Radio 13	Rompe bloques (260)	200 × 2

Todas las armas de bala admiten headshot, que mata en el acto.

Clases

Clase	Rol	Vida	Velocidad	Arma inicial	Granadas
Soldado	Asalto	100	14	Fusil de Cerrojo M1	1
Pyro	Corta distancia	120	14,5	Lanzallamas	0

Mejoras de la tienda: Chaleco (+30 de vida, 300), Botas (+12 % de velocidad, 250), Cargadores extra (+50 % de munición, 200), Bloques de caramelo ×8 (150). Las granadas extra tienen un tope de +4.

Comunicación de red

Hay 26 eventos remotos en `ReplicatedStorage.Remotes`, sin `RemoteFunction`. El servidor valida tipo y fase en todo lo que llega del cliente; el estado continuo (fase, reloj, progreso, munición, dinero) viaja por atributos y no por eventos.

Remote	Dirección	Lo atiende	Propósito
`Player.Deploy`	Cliente → servidor	CharacterSystem	Entrar a jugar con bando y clase elegidos (sin bando válido, equilibra automáticamente)
`Player.ReturnToMenu`	Cliente → servidor	CharacterSystem	Salir al menú y liberar el lugar para un bot
`Player.SelectClass`	Cliente → servidor	CharacterSystem	Cambiar de clase (inmediato en el interludio, al reaparecer en ronda; máximo 1 cada 0,5 s)
`Player.SetReady`	Cliente → servidor	CharacterSystem	Marcarse listo en el interludio
`Player.IntroSeen`	Cliente → servidor	ProfileSystem	Guardar que el jugador vio la intro
`Combat.Fire`	Cliente → servidor	CombatSystem	Disparar con la lista de ángulos de cada perdigón

Remote	Dirección	Lo atiende	Propósito
`Combat.Aim`	Cliente → servidor (no confiable)	CombatSystem	Ángulo de apuntado, replicado como atributo del modelo
`Combat.Reload`	Cliente → servidor	CombatSystem	Pedir recarga
`Combat.Grenade`	Cliente → servidor	CombatSystem	Lanzar granada en un ángulo
`Building.Place`	Cliente → servidor	BuildSystem	Colocar un bloque en columna y fila
`Building.RemoveBlock`	Cliente → servidor	BuildSystem	Quitar un bloque propio y recuperarlo
`Shop.Buy`	Cliente → servidor	LoadoutSystem	Comprar un artículo (solo en `Prep` o `Waiting`)
`Shop.Equip`	Cliente → servidor	LoadoutSystem	Equipar un arma ya comprada
`Combat.Projectile`	Servidor → todos	ProjectileRenderer	Dibujar los proyectiles de un disparo
`Combat.Hit`	Servidor → todos	HUDController	Daño recibido: hitmarker, números, viñeta
`Combat.KillFeed`	Servidor → todos	HUDController	Línea del killfeed
`Combat.GrenadeThrown`	Servidor → todos	ProjectileRenderer	Crear o retirar el visual de una granada
`Combat.Explosion`	Servidor → todos	ProjectileRenderer, CinemaController	Explosión en posición y radio
`Combat.Structure`	Servidor → todos	ProjectileRenderer	Un bloque o depósito fue destruido
`Combat.Knockback`	Servidor → un jugador	InputController	Empuje aplicado por el cliente dueño de la física

Remote	Dirección	Lo atiende	Propósito
`Round.Event`	Servidor → todos	HUDController, CinemaController	Cambios de fase: `Prep`, `Round`, `OneMinute`, `RoundEnd`, `MatchEnd` con datos del final
`Round.State`	—	—	Declarado, sin uso actual
`Objectives.Event`	Servidor → todos	HUDController	Avisos de misión (VIP, colina, radio, bandera, depósito)
`Shop.Notice`	Servidor → un jugador	HUDController	Avisos de dinero, compra o error
`UI.Intercom`	Servidor → equipo o jugador	IntercomController	Clave de línea del comandante
`UI.Toast`	—	—	Declarado, sin uso actual

Atributos replicados

Dónde	Atributos principales
`MatchState`	`Phase`, `PhaseEnd`, `PhaseDuration`, `Round`, `Mission`, `Missions`, `WinsUSA`, `WinsVietnam`, `ProgressUSA`, `ProgressVietnam`, `LastWinner`, `MatchWinner`, `TotalDeaths`, contadores de señales y capturas
`MatchState.Combatants.<clave>`	`Name`, `Team`, `Class`, `IsBot`, `Kills`, `Deaths`, `Headshots`, `Score`
`Player`	`Deployed`, `Ready`, `Class`, `RespawnAt`, `Money`, `Owned`, `EquipSoldier`, `EquipPyro`, `Vest`, `Boots`, `Mags`, `GrenadeBonus`, `BlocksLeft`, `SeenIntro`, `ProfileLoaded`
Modelo del personaje	`Key`, `Team`, `Class`, `DisplayName`, `IsBot`, `Aim`, `Weapon`, `Ammo`, `MaxAmmo`, `Grenades`, `Reloading`, `Burning`, `BaseSpeed`, `VIP`, `CarryingFlag`, `LastHitBy`

Dónde	Atributos principales
Marcadores de objetivo	`Kind`, `Team`, `State`, `Owner`, `Contested`, `Active`, `HackUSA`, `HackVietnam`, `Health`, `MaxHealth`

La clave de un combatiente es `P<UserId>` para humanos y `B<n>` para bots. El tiempo de fase se sincroniza con el reloj del servidor (`GetServerTimeNow`).

Flujo de una partida

Una partida va de `Waiting` a `MatchEnd` pasando hasta tres veces por `Prep → Round → RoundEnd`, y termina antes si un equipo gana dos rondas. Si en cualquier momento no queda ningún humano desplegado, la partida se corta y vuelve a `Waiting`.

1. **Entrada.** ProfileSystem carga el perfil. IntroController reproduce una cinemática al azar (según `IntroData.PlayOnJoin`) y avisa `IntroSeen`. MenuController muestra el menú con la cámara en paneo.
2. **Despliegue.** El jugador elige bando y clase; el cliente envía `Deploy`. CharacterSystem asigna equipo, registra al combatiente en LoadoutSystem, dispara `Deployed` y hace aparecer al personaje.
3. **Waiting.** MatchSystem espera al primer jugador desplegado, sortea el orden de las misiones (sin repetir) y reinicia dinero, estadísticas y bloques construidos.
4. **Prep (60 s).** Se activan las barreras entre mitades, se recargan los bloques y todos reaparecen en su base. Se puede construir (B), comprar (T) y marcar listo (F). Los bots compran y construyen a los 3 s. El comandante da la bienvenida (solo en la ronda 1) y explica la misión. Si todos los humanos están listos, el reloj salta a 3 s.
5. **Round (300 s).** Se quitan las barreras, ObjectiveSystem arranca la misión, todos reaparecen y el daño se habilita. Faltando 60 s se emite `OneMinute`. La ronda termina cuando un equipo completa el objetivo o se acaba el tiempo; en ese caso gana quien tenga más progreso y, si hay empate exacto, nadie.
6. **RoundEnd (9 s).** Se suma la victoria, se paga 600 al ganador y 300 al resto, y el comandante comenta el resultado. Si un equipo llegó a 2 victorias, se pasa al final.
7. **MatchEnd (58 s).** Se calcula el ganador y el MVP por puntaje. El servidor manda en `Round.Event` los 3 mejores de cada bando, y FinaleController reproduce el final del bando ganador seguido de los resultados.
8. **Reinicio.** MatchSystem vuelve a `Waiting` y repite el ciclo; los bots se retiran en `Waiting`.

Misiones

Misión	Cómo se gana	Progreso visible	Recompensas
Asesinato	Matar al VIP enemigo, elegido al azar, con 260 de vida	Daño hecho al VIP enemigo (tope 99 % hasta la baja)	300 a quien mata al VIP
Rey de la Colina	Controlar sola la zona central (x −34 a 34) durante 80 s acumulados; más aliados la llenan hasta un 50 % más rápido	Tiempo de control	40 por cada 10 % a quienes están en la colina
Intercepción (Radio)	Hackear 3 señales; cada una aparece en uno de 9 puntos y requiere 7 s sin enemigos cerca	Señales + hackeo en curso	150 a quienes hackean
Robo de Bandera	Llevar la bandera enemiga a la propia base 2 veces; el portador va 10 % más lento	Capturas + 15 % mientras se porta	300 por captura, 50 por recuperar la propia
Demolición	Destruir el depósito enemigo (1.600 de vida)	Daño al depósito	50 cada 200 de daño, 300 por destruirlo

Una bandera caída vuelve sola a su base a los 20 s. Para pruebas en Studio, `MatchState.SkipPhase = true` termina la fase actual y `MatchState.ForceMissions` fija el orden de misiones.

Pendientes conocidos y notas de mantenimiento

El juego está completo de punta a punta; lo que queda son ajustes de contenido, limpieza del árbol y deuda técnica menor.

Pendientes

- [ ] Cargar el ID del cover de la canción de "Una vida dulce" en `IntroData.LifeSong`; mientras esté vacío suena el tema de respaldo.
- [ ] Limpiar `SoundService`: hay 28 `SoundGroup` duplicados (14 pares `IntroMusic` / `IntroSfx`).
- [ ] Quitar o empezar a usar los remotes `Round.State` y `UI.Toast`, que hoy nadie escucha.

- [ ] Decidir si `IntroData.PlayOnJoin` queda en `Always`; con ese valor el registro de intro vista en el DataStore no cambia nada.

Notas de mantenimiento

- **Un solo mapa cableado.** CharacterSystem, BuildSystem, BotSystem y NavGrid apuntan directo a `WarGummyMap01`. Agregar mapas requiere centralizar esa referencia.
- **Coordenadas fijas en código.** Puntos de radio, bases de bandera (±241), depósitos (±249,5), posición del VIP y destinos que valida la construcción están escritos en los sistemas, no en el mapa. Las carpetas del mapa pensadas para eso (`Objectives.VIP`, `Zones.BuildZones`, `Zones.ShopZones`, `CameraBounds`) están vacías.
- **Lógica de navegación duplicada.** BotSystem y NavGrid repiten las mismas reglas de salto. Un cambio en una debe copiarse en la otra o los bots quedarán atascados en construcciones que NavGrid aceptó.
- **Persistencia mínima.** Solo se guarda si el jugador vio la intro. Dinero, estadísticas y compras se reinician en cada partida por diseño.
- **Orden de arranque del servidor.** Los sistemas se inicializan sin orden garantizado; ningún `Init` debe depender de otro sistema.
- **Validación de entradas.** Disparo, granada y cambio de clase tienen límites de frecuencia en el servidor. Compra, colocación de bloques y `SetReady` dependen solo de la fase; conviene agregarles un límite de frecuencia.
- **Módulos grandes.** `IntroCine.Engine` (~2.000 líneas), `FinaleController` (~1.500) e `IntroCine.Life` (~1.200) concentran la mayor parte del código de presentación.
- **Carpetas sin contenido.** `Config`, `Animations`, `Assets.Effects`, `Assets.SurfaceGui` y las plantillas de bloques, armas, ítems y NPC en `ServerStorage` existen pero están vacías.
- **Material fuera de juego.** `ServerStorage.Debug` (MapBuilder, TestSystem, prototipos) y `ServerStorage.Backups` (intro anterior) no se ejecutan; `TestSystem` sirve de plantilla para un sistema nuevo con `Init` y `Start`.